

# Impedance mismatch de dades vectorials

## CBDE – Laboratori 1: PostgreSQL, Chroma i pgvector

Sergi Galán Soler

Joaquim Fuentes Pou

2026-09-28

**Repositori (codi i chunk de dades):** <https://github.com/sgalans/CBDE---L1>

**Dades i metodologia.** De bookCorpus (HuggingFace, en *streaming*) s'han extret 10.000 frases netes i sense duplicats, agrupades en 500 chunks de 20 frases consecutives (`data/chunks.parquet`). Els tres sistemes carreguen exactament el mateix split per frases (`data/sentences.parquet: sentence_id, chunk_id, pos, text`). Els embeddings es generen amb `all-MiniLM-L6-v2` (384 dimensions), l'última capa del qual (`Normalize`) retorna vectors de **norma 1** (verificat sobre els vectors desats). Les mètriques són **L2** i **cosinus**, i el top-2 exclou sempre la pròpia frase. Els temps es mesuren amb `time.perf_counter` en un únic portàtil (Intel Core i7-13700H, 16 GB, PostgreSQL 16 a Docker Desktop). La unitat de mesura és l'operació: **un lot** en les insercions i **una consulta** en la cerca. Les estadístiques (mínim, màxim, mitjana, desviació) es calculen sobre totes les operacions mesurades, i  $n = \text{lots (o consultes)} \times \text{repeticions}$ ; cada taula indica quantes repeticions s'han fet. La generació d'embeddings es mesura un sol cop per script, perquè és cara; tot i així, el mateix model a la mateixa màquina ha donat entre 1.591 i 1.654 ms per lot segons l'execució, cosa que indica el soroll entre execucions i obliga a llegir amb cautela les diferències petites entre sistemes. Abans de cronometrar es fa un *warm-up* (càrrega del model, primera execució de les consultes) que no es compta.

| ID   | Frase de consulta                                                                                                                                                                                       |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 654  | <i>jasons face grinned in a seriously weird way that screamed , weirdo !</i>                                                                                                                            |
| 1114 | <i>it must have been some type of faery because i could see long , pointed ears sticking up from the sides of its head .</i>                                                                            |
| 2025 | <i>it involves lots of smart remarks and bad decisions involving life or death .</i>                                                                                                                    |
| 3759 | <i>he was squinting at the thing with extreme concentration , like a successful young stock broker in an armani suit instead of a missionary in cut-off khaki shorts that really needed a haircut .</i> |
| 4281 | <i>she took one last glance at wara but did n't seem to be concerned as she sauntered into the back room .</i>                                                                                          |
| 5250 | <i>wara bit her lip , then strummed a chord .</i>                                                                                                                                                       |
| 6228 | <i>for that , i offer my life now in jihad , against the injustices of the world and against the unrighteousness in my own heart .</i>                                                                  |
| 7142 | <i>trae watched in horror as the buicks dark tinted windows began to lower .</i>                                                                                                                        |
| 8754 | <i>but three misses , we go home and im two hundred richer .</i>                                                                                                                                        |
| 9104 | <i>theyd always wear gloves , and mix it fifty parts water to one part acid .</i>                                                                                                                       |

**Taula 1.** Les 10 frases de consulta (`data/queries.json`), idèntiques als tres sistemes. ID = `sentence_id`.

## 1. PostgreSQL

### 1.1. Decisions de disseny

**Split per frases.** L'enunciat demana dividir els chunks en frases en carregar-los. Aquesta divisió es fa una sola vegada, en preparar les dades, perquè sigui exactament la mateixa a tots els sistemes; cada frase conserva el chunk d'on ve (`chunk_id`) i la seva posició (`pos`), de manera que els chunks es poden reconstruir des de la BD.

**Tipus i esquema.** El text va a `sentences(sentence_id INTEGER PK, chunk_id INTEGER, pos SMALLINT, text TEXT)`: `TEXT` té el mateix emmagatzematge que `VARCHAR(n)` sense comprovar longituds. Els embeddings van a una taula a part, `sentence_embeddings(sentence_id PK/FK, embedding REAL[]): REAL (float4)` és la precisió que produeix el model

(`float8[]` doblaria la mida sense guany), i una taula separada fa que desar-los sigui un `INSERT` pur; un `UPDATE` sobre `sentences` escriuria una versió nova de cada fila i en deixaria una de morta. Aquí apareix el primer `mismatch: REAL[]` és un array genèric. PostgreSQL no sap que és un vector de 384 dimensions, no en controla la mida i no té operadors de distància ni índexs per a ell.

**Inserció per lots.** Cada lot és un únic `INSERT` multi-fila (`execute_values`) i un `COMMIT`, de manera que les crides a la BD són exactament el nombre de lots. S'ha provat un grid de mides (1 a 2000; la mida 1 és el *baseline* fila a fila) i s'ha triat **1000**, el punt on la corba s'aplanava: més enllà, els guanys són marginals (centèsimes de segon per al text, i per als embeddings al límit del soroll ja des de ~500 files). Lots més grans només farien transaccions i missatges més grans (a **P1**, uns quants MB de SQL per lot) sense cap benefici.

**El viatge dels vectors.** **P1** llegeix el text de la BD (1 `SELECT`, 12 ms), genera els embeddings a Python i els torna a escriure. Com que PostgreSQL no té cap tipus que rebí un array numpy, **cada float viatja com a text i el servidor l'ha de tornar a analitzar**. L'adaptació per defecte de psycopg2 genera `ARRAY[...]` amb un literal `float8` per component (~8 MB de SQL per lot de 1000 vectors); enviar cada vector com un sol literal `'{...}'::real[]` és 2,8 vegades més ràpid, i és el que fa **P1**. Amb `repr` el valor retornat és bit a bit idèntic al generat (verificat).

**Distàncies dins la BD.** Com que no hi ha operadors vectorials, les dues mètriques s'han hagut d'escriure com a funcions SQL sobre `unnest(a, b)`, i el top-k com una funció PL/pgSQL (`top_k_similar`) que llegeix el vector de la consulta, exclou la pròpia frase amb `WHERE` i fa `ORDER BY distància LIMIT k`. Python envia un identificador i rep dues files: els vectors no surten mai de la BD. Els càlculs es fan en `float8`, perquè elevar al quadrat un `float4` molt petit fa que PostgreSQL llanci un error d'*underflow* en lloc d'arrodonir a 0. La cerca és **exacta** (força bruta sobre les 10.000 files).

**Línies i crides.** El codi que parla amb la BD ocupa 27 línies a **P0**, 40 a **P1** i 71 a **P2**, la majoria SQL de les funcions de distància que el sistema no ofereix. La càrrega fa un `INSERT` i un `COMMIT` per lot, i la cerca fa una crida per consulta i mètrica (la comparació entre sistemes és a la Discussió).

## 1.2. [PQ1]

**Són estables els temps d'inserció de text i embeddings?** Amb lots, sí: a la mida triada el coeficient de variació per lot és baix (taules següents). El menys estable és el fila a fila (CV del temps total 26 % entre càrregues), perquè 10.000 crides acumulen tot el soroll del sistema.

| Mida de lot | Lots ( <code>INSERT</code> ) | Text: total (s) | desv. entre càrregues | Embeddings: total (s) | desv. entre càrregues |
|-------------|------------------------------|-----------------|-----------------------|-----------------------|-----------------------|
| 1           | 10000                        | 26,585          | 6,982                 | 27,779                | 6,733                 |
| 10          | 1000                         | 3,590           | 0,101                 | 6,800                 | 0,837                 |
| 50          | 200                          | 0,753           | 0,002                 | 3,810                 | 0,035                 |
| 100         | 100                          | 0,458           | 0,032                 | 3,410                 | 0,026                 |
| 500         | 20                           | 0,135           | 0,004                 | 2,926                 | 0,011                 |
| <b>1000</b> | 10                           | 0,097           | 0,003                 | 2,784                 | 0,037                 |
| 2000        | 5                            | 0,083           | 0,003                 | 2,733                 | 0,029                 |

**Taula 2.** Temps **total** de càrrega de les 10.000 frases per mida de lot: mitjana i desviació entre 3 càrregues completes. En negreta, la mida triada.

| Operació                         | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV   |
|----------------------------------|----|----------|----------|--------------|------------|------|
| Emmagatzematge del text (P0)     | 30 | 8,5      | 11,9     | 9,7          | 0,6        | 6 %  |
| Generació d'embeddings (P1)      | 10 | 1.252,1  | 1.993,1  | 1.654,2      | 223,6      | 14 % |
| Emmagatzematge d'embeddings (P1) | 30 | 268,8    | 291,4    | 278,4        | 6,1        | 2 %  |

**Taula 3.** Temps **per lot** de 1000 frases; n = lots mesurats (10 lots × 3 càrregues per a l'emmagatzematge, 10 lots per a la generació). CV = desviació / mitjana.

Les conclusions són dues. (1) **Per al text, el cost és per crida, no per fila**: cada crida costa ~2,7 ms (anada i tornada + `COMMIT`), i agrupar-les redueix el temps de 26,6 s a 0,10 s (274×). (2) **Per als embeddings, a partir de ~500 files la**

**mida de lot deixa d'importar:** el coll d'ampolla ja no són les crides sinó convertir 384 floats per vector a text i tornar-los a analitzar. Desar-los costa 29 vegades més que desar el text de les mateixes files. Una part és inherent (384 valors per frase), però una altra és *impedance mismatch*: com que `REAL[]` no és un tipus vectorial, el vector viatja com a text, i la manera de serialitzar-lo ja canvia el cost 2,8 vegades (vegeu *El viatge dels vectors*). La generació (16,5 s en total) és el cost més gran, però és un cost de Python que no depèn de la base de dades.

**Són estables els temps de consulta? Hi ha diferències entre mètriques?** Sí, són estables (CV 14 % per a L2 i 13 % per al cosinus): cada consulta fa exactament la mateixa feina, recórrer les 10.000 files i operar 384 components per fila, sigui quina sigui la frase. Per això, les variacions (màxim de 406 ms) s'atribueixen a interferències puntuals del sistema i no a diferències entre consultes.

| Mètrica                             | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV   |
|-------------------------------------|----|----------|----------|--------------|------------|------|
| L2                                  | 30 | 242,2    | 405,8    | 277,1        | 38,3       | 14 % |
| Cosinus                             | 30 | 274,8    | 400,4    | 319,8        | 40,2       | 13 % |
| Cosinus unitari ( $1 - a \cdot b$ ) | 30 | 219,2    | 315,6    | 243,0        | 25,4       | 10 % |

**Taula 4.** Temps **per consulta** del top-2; n = 10 consultes × 3 rondes. El cosinus unitari és una variant de referència, no una tercera mètrica.

Les dues mètriques donen **el mateix top-2 en 10/10 consultes**. No és casualitat: amb vectors de norma 1,  $\|a - b\|^2 = 2 - 2 \cos(a, b)$ , és a dir, la distància L2 és una funció creixent de la distància cosinus i ordenen igual. A les dades,  $|L2^2 - 2 d_{cos}| \leq 1 \cdot 10^{-7}$ . L'única diferència és el temps: el cosinus és un 15 % més lent perquè calcula tres sumes (producte escalar i dues normes) en lloc d'una.

### Quines tècniques de PostgreSQL (sense pgvector) millorarien el rendiment?

- **Inserció.** `COPY` en lloc d'`INSERT` (mesurat: un 22 % menys de temps amb lots de 1000, perquè el servidor no ha d'analitzar un `INSERT` multi-fila; el guany és limitat perquè el nombre de crides és el mateix); `COPY binari` per als embeddings, que evitaria justament la conversió float→text; taules `UNLOGGED` o `synchronous_commit = off` durant la càrrega; crear la clau primària i la forana després de carregar; i un sol `COMMIT` final.
- **Estructura de dades.** Guardar vectors normalitzats (o la norma precalculada) permet reduir el cosinus a  $1 - a \cdot b$ : ho hem mesurat i és un 24 % més ràpid amb el mateix resultat. Un `bytea` amb els 1.536 bytes crus ocuparia menys i evitaria el text, però la BD no el podria llegir sense una funció en C.
- **Consulta.** Funcions en C en lloc de SQL interpretat amb `unnest`; paral·lelisme (`PARALLEL SAFE + max_parallel_workers_per_gather`); i resoldre les 10 consultes en una sola crida (`CROSS JOIN LATERAL`). Com a índex, l'extensió `cube` amb GiST permet cercar el veí més proper per distància euclidiana, però està limitada a 100 dimensions (caldrà reduir-ne la dimensionalitat, perdent precisió). Cap índex estàndard (B-tree, GIN, GiST sobre arrays) accelera la distància a un vector arbitrari: aquesta és la mancança que resol pgvector.

## 2. Chroma

### 2.1. Decisions de disseny

**Model de dades i client.** A Chroma la unitat és el registre d'una col·lecció: un identificador, **un vector** i, opcionalment, el document (el text) i unes metadades. El vector és l'element central i el text n'és un atribut, just a l'inrevés que a PostgreSQL. Cada registre guarda com a metadades `sentence_id`, `chunk_id` i `pos`, de manera que el split és el mateix que a PostgreSQL. S'ha fet servir el client integrat (`PersistentClient`): Chroma s'executa dins el mateix procés de Python i desa les dades en un directori local, sense passar per la xarxa. Cal tenir-ho present en comparar temps amb PostgreSQL, on cada crida és una anada i tornada per TCP.

**Una col·lecció per mètrica.** La distància es fixa en crear la col·lecció (`configuration={"hnsw": {"space": ...}}`), perquè determina com es construeix l'índex HNSW, i no es pot triar a cada consulta. Per tenir L2 i cosinus calen dues col·leccions (`sentences_l2` i `sentences_cosine`) amb les mateixes dades: el doble d'emmagatzematge i el doble d'índexs. A PostgreSQL, en canvi, una sola taula serveix per a qualsevol funció de distància.

**C0: el text no es pot inserir sense vector.** Sense funció d'embeddings, `add(documents=...)` llança un error; amb la funció per defecte (una versió ONNX del mateix `all-MiniLM-L6-v2`), és el mateix `add()` qui calcula els vectors. S'ha fet servir aquesta segona opció, que és l'ús natural de Chroma, amb el mateix grid de mides de lot que a PostgreSQL i un *warm-up* previ del model ONNX. La conseqüència és que el temps d'"inserir el text" **inclou per força la generació dels**

**embeddings:** no es pot comparar amb P0 sol, sinó amb P0 més la generació de P1. Aquesta comparació és aproximada, perquè també enfronta dos entorns d'execució del model (ONNX a Chroma, PyTorch a P1). A la càrrega oficial, que omple les dues col·leccions, els embeddings es calculen dues vegades, una per col·lecció. Per limitar el temps d'execució, el grid de mides es fa només a la col·lecció L2 i amb una sola càrrega per mida; a la mida triada (1000), en canvi, la col·lecció L2 es carrega 3 vegades, com a PostgreSQL, per poder comparar l'estabilitat entre càrregues.

**C1: substituir els vectors pels del nostre model.** Com a P1, el script llegeix els documents de la mateixa base de dades (un `get()`), genera els embeddings amb el model compartit amb PostgreSQL i els desa amb `update(ids, embeddings)` per lots a cada col·lecció. Després de C1, Chroma cerca sobre els mateixos embeddings (PyTorch) que PostgreSQL i pgvector; els que havia generat C0 amb ONNX hi coincidien fins a  $2 \cdot 10^{-7}$ , però no bit a bit (a la col·lecció de cosinus, a més, Chroma els arrodoneix en normalitzar-los, vegeu [CQ1]). La diferència amb P1 és que Chroma rep directament l'array de numpy: no hi ha cap conversió float  $\rightarrow$  text  $\rightarrow$  float.

**C2: consulta amb índex i sense cerca per identificador.** A la versió utilitzada (chromadb 1.5.9), `query()` necessita un vector o un text de consulta: el seu paràmetre `ids` només restringeix entre quins registres es busca, no permet buscar els veïns d'un registre pel seu id (comprovat). Cada consulta són, doncs, dues crides: `get()` per obtenir el vector desat de la frase i `query()` amb `n_results = 2` i el filtre de metadades `where={"sentence_id": {"$ne": id}}`, que exclou la pròpia frase. La cerca passa per l'índex HNSW i és **aproximada**; els resultats exactes de P2 serveixen de referència per mesurar-ne el *recall*. A més, l'espai l2 de Chroma retorna la distància euclidiana **al quadrat**.

**Línies i crides.** C0, C1 i C2 ocupen 23, 17 i 12 línies de sistema: no cal esquema ni SQL, però la càrrega oficial es fa dues vegades (una per col·lecció) i cada consulta necessita dues crides (`get` + `query`). No hi ha COMMIT: Chroma no té transaccions explícites.

## 2.2. [CQ1]

**Són estables els temps d'inserció de text i embeddings?** Sí. A la mida triada, cada lot d'`add()` varia poc (CV per lot a les taules següents) i el temps total de les 3 càrregues oficials a L2 és de 141,3 s, amb una desviació de 0,5 s entre càrregues (< 1 %). La raó és que l'`add()` és gairebé tot càlcul: **el 98 % del temps d'un lot és generar els embeddings ONNX** (fila *Només embedding*), una feina regular que depèn del nombre de frases i no de l'estat del sistema.

| Mida de lot | Crides <code>add()</code> | Total (s) | Per <code>add()</code> (ms) | × PostgreSQL |
|-------------|---------------------------|-----------|-----------------------------|--------------|
| 1           | 10000                     | 1.523,9   | 152,4                       | 35,3         |
| 10          | 1000                      | 274,7     | 274,7                       | 13,6         |
| 50          | 200                       | 171,1     | 855,3                       | 9,9          |
| 100         | 100                       | 156,8     | 1.568,5                     | 9,2          |
| 500         | 20                        | 142,4     | 7.121,8                     | 8,5          |
| <b>1000</b> | 10                        | 141,1     | 14.109,5                    | 8,5          |
| 2000        | 5                         | 139,8     | 27.955,8                    | 8,4          |

**Taula 5.** C0: temps **total** de càrrega de les 10.000 frases a la col·lecció L2 per mida de lot, dins el grid (1 càrrega per mida; les 3 càrregues oficials a la mida triada són a la Taula següent i al text). Inclou el càlcul dels embeddings ONNX dins l'`add()`. × PostgreSQL: vegades el temps de fer el mateix a PostgreSQL, és a dir, P0 a la mateixa mida més la generació d'embeddings de P1. En negreta, la mida triada.

| Operació                                               | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV    |
|--------------------------------------------------------|----|----------|----------|--------------|------------|-------|
| <code>add()</code> text + embedding ONNX, L2 (C0)      | 30 | 13.952,0 | 14.315,4 | 14.131,2     | 96,6       | 1 %   |
| <code>add()</code> text + embedding ONNX, Cosinus (C0) | 10 | 14.090,3 | 14.309,8 | 14.206,3     | 63,9       | < 1 % |
| Només embedding ONNX, referència (C0)                  | 10 | 13.617,8 | 13.908,0 | 13.784,4     | 105,9      | 1 %   |
| Generació d'embeddings, nostre model (C1)              | 10 | 1.255,7  | 1.959,3  | 1.636,7      | 219,4      | 13 %  |
| <code>update()</code> embeddings, L2 (C1)              | 30 | 258,5    | 334,6    | 297,0        | 17,5       | 6 %   |

| Operació                                                   | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV  |
|------------------------------------------------------------|----|----------|----------|--------------|------------|-----|
| <code>update()</code> embeddings, Cosinus (C1)             | 30 | 249,4    | 374,6    | 284,8        | 23,9       | 8 % |
| Ref.: <i>emmagatzematge d'embeddings a PostgreSQL (P1)</i> | 30 | 268,8    | 291,4    | 278,4        | 6,1        | 2 % |

**Taula 6.** Temps **per lot** de 1000 frases a Chroma; n = lots mesurats (10 per càrrega: 3 càrregues a L2 i 1 a cosinus a C0; a C1, 1 generació i 3 `update()` complets per col·lecció; la fila *Només embedding* és una passada de la funció ONNX sobre els mateixos 10 lots, sense desar res). L'última fila és la de PostgreSQL, per comparar.

D'aquí surten tres conclusions. (1) **Els lots ajuden més a la càrrega de Chroma que a la de PostgreSQL, però pel model que s'executa dins l'`add()`, no per la base de dades.** A Chroma, passar de fila a fila (25,4 min) a lots de 1000 (141,1 s, al grid) redueix el temps 11 vegades. L'equivalent a PostgreSQL (P0 a la mateixa mida més la generació de P1) passa de 43,1 s a 16,6 s, només 2,6 vegades, perquè a P1 la generació es fa sempre per lots, sigui quina sigui la mida del lot d'inserció. A Chroma, en canvi, el model s'executa dins l'`add()` amb el mateix lot, i amb una sola frase l'ONNX és molt ineficient: un `add()` d'una frase costa 152 ms, 57 vegades un `INSERT` d'una fila. (2) **La mètrica afecta poc la inserció:** amb lots de 1000, la col·lecció de cosinus (una sola càrrega) difereix un 1 % de la d'L2, una diferència petita i propera a la variació entre lots. (3) **Desar els vectors no és més barat que a PostgreSQL:** tot i rebre l'array de numpy directament, sense passar per text, un `update()` de 1000 vectors triga 297 ms, un 7 % més lent que l'`INSERT` de `REAL[]` de P1. Cal tenir en compte que Chroma, a més d'escriure el vector, l'ha d'inserir a l'índex HNSW; dins l'`add()`, la part que no és embedding costa 347 ms per lot.

**Són estables els temps de consulta? Hi ha diferències entre mètriques?** Sí (CV 4 % per a L2 i 3 % per al cosinus). Una consulta triga uns 13 ms, **entre 21 (L2) i 25 (cosinus) vegades menys que a PostgreSQL**. Però el 93 % d'aquest temps és el filtre de metadades (vegeu les millores): sense filtre, obtenir el vector i cercar a l'índex HNSW triga 0,92 ms, unes **302 vegades menys que la força bruta de P2**. Aquesta comparació aïlla millor el guany atribuïble a l'índex vectorial. La xarxa no sembla explicar la major part de la diferència: fins i tot un `INSERT` d'una fila amb el seu `COMMIT` (mida de lot 1 a P0), que inclou l'anada i tornada, és només un 1 % d'una consulta de P2.

Tot i que l'índex HNSW és aproximat, en les 10 consultes ha recuperat **tots els veïns exactes** de P2 (*recall* del 100 % en totes dues mètriques). És una mostra petita (20 veïns), però té una explicació: amb el valor per defecte `ef_search = 100`, cada cerca explora 100 candidats per retornar-ne 2. No recorre tots els vectors, però aquest marge tan ampli respecte dels 2 que es demanen fa molt probable trobar els veïns exactes. És també el marge que es podria reduir per guanyar velocitat.

| Mètrica                       | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV   | Recall vs P2 |
|-------------------------------|----|----------|----------|--------------|------------|------|--------------|
| L2                            | 30 | 12,15    | 13,99    | 12,90        | 0,49       | 4 %  | 100 %        |
| Cosinus                       | 30 | 12,30    | 13,94    | 12,83        | 0,40       | 3 %  | 100 %        |
| L2, sense filtre (k + 1)      | 30 | 0,71     | 1,48     | 0,92         | 0,18       | 19 % | 100 %        |
| Cosinus, sense filtre (k + 1) | 30 | 0,65     | 1,51     | 0,87         | 0,16       | 18 % | 100 %        |

**Taula 7. C2:** temps **per consulta** del top-2 (`get()` + `query()`); n = 10 consultes × 3 rondes. *Recall:* part dels veïns exactes de P2 que retorna l'índex HNSW. Les files *sense filtre* són una variant de referència: demanen k + 1 veïns i descarten la pròpia frase a Python; retornen els mateixos veïns que amb el filtre, i per tant el mateix *recall*.

Com a PostgreSQL, les dues col·leccions retornen **el mateix top-2 en 10/10 consultes** i les distàncies compleixen la identitat ( $|d_{l2} - 2d_{cos}| \leq 5 \cdot 10^{-7}$ , on  $d_{l2}$  és el valor que retorna Chroma, que ja és  $\|a - b\|^2$ ). Els temps tampoc no difereixen: l'índex fa la mateixa feina amb qualsevol mètrica. Un detall: l'espai **cosine torna a normalitzar els vectors en desar-los**, i 2.754 dels 10.000 vectors no s'hi guarden exactament com es van enviar (diferència màxima  $1 \cdot 10^{-8}$ ).

**Es pot mesurar per separat la inserció del text i la creació d'embeddings? No de manera nativa.** Chroma no accepta un registre sense vector, i amb la funció d'embeddings per defecte el vector es calcula dins el mateix `add()`: el temps "d'inserir text" de C0 inclou la generació. Només s'ha pogut aproximar la separació per fora: mesurant la funció

d'embeddings sola (98 % de l'`add()`) i, a **C1**, generant els vectors per separat i substituint-los amb `update()`, cosa que obliga a escriure dues vegades cada vector. La conclusió és l'invers de PostgreSQL: allà el vector era un afegit al text; aquí el text és un atribut del vector, i és el text el que no té una existència independent.

### Quines tècniques de Chroma millorarien el rendiment?

- *Generar els embeddings fora i passar-los a l'`add()`.* La funció per defecte (ONNX) és 8,4 vegades més lenta que el mateix model amb PyTorch a **C1** (els vectors coincideixen fins a  $2 \cdot 10^{-7}$ , així que la diferència és de l'entorn d'execució de Chroma, no del model), i a més es calcula un cop per col·lecció. Passant `embeddings=` s'evitarien totes dues coses.
- *Una sola col·lecció.* Amb vectors normalitzats, L2, cosinus i producte escalar ordenen igual: una sola col·lecció serviria les dues mètriques i estalviaria la meitat d'emmagatzematge i d'inserció.
- *Evitar el filtre de metadades.* Demanar un veí més ( $k + 1$ ) i descartar la pròpia frase a Python dona el mateix resultat (20/20) i redueix el temps de consulta un 93 %, és a dir, és unes 14 vegades més ràpid: el filtre `where` afegeix un pas sobre les metadades a una cerca que, sense ell, només recorre l'índex HNSW.
- *Paràmetres d'HNSW.* `ef_search`, `max_neighbors` i `ef_construction` permeten intercanviar temps per *recall*, que aquí ja és del 100 %, i per tant hi ha marge per abaratir la construcció i la cerca; `batch_size` i `sync_threshold` controlen cada quants vectors s'actualitza i es persisteix l'índex, i afecten el cost de la inserció. Per a càrregues concurrents, el mode client-servidor. En canvi, lots més grans no ajudarien: amb el model dins l'`add()`, més enllà de ~500 frases el temps ja no baixa.

## 3. pgvector

### 3.1. Decisions de disseny

**Mateix motor, un tipus nou.** pgvector s'activa amb `CREATE EXTENSION vector` en una base de dades separada (`cbde_pgvector`), perquè la part de PostgreSQL pur no hi tingui accés. Per al text no canvia res: **G0 reutilitza el codi de P0** (mateixa taula, tipus i `INSERT` per lots) i no té cap línia de sistema pròpia; el temps de càrrega difereix un 6 % del de P0. pgvector estén el model relacional en lloc de substituir-lo.

**vector(384) en lloc de REAL[].** És un tipus vectorial de debò: la base de dades en comprova la dimensió i ofereix operadors de distància (`<->` L2, `<=>` cosinus) i índexs. Els vectors s'envien amb l'adaptador oficial (`pgvector.psycopg2`), que converteix cada array de numpy en un **literal de text** '`[x1,x2,...]`': el conductor continua parlant en text, com a P1, però el servidor l'analitza cap a un tipus natiu. Amb lots de 1000, desar-los costa 2,47 s, un 11 % menys que el `REAL[]` de P1, i el vector es recupera bit a bit idèntic.

**Un índex HNSW per mètrica, creat després de carregar.** Com a Chroma, l'índex depèn de la distància (classes d'operadors `vector_l2_ops` i `vector_cosine_ops`), però aquí **les dades es desen una sola vegada** i la taula té dos índexs. Es construeixen després de la càrrega, com recomana pgvector (Chroma, en canvi, actualitza l'índex dins de cada `add()`). Construir-los triga 0,71 s (L2) i 0,70 s (cosinus), i cadascun ocupa 20,5 MB, més que la taula sencera (16,4 MB).

**Consulta: una sola crida SQL.** `ORDER BY embedding <-> (SELECT embedding ... WHERE sentence_id = q) LIMIT 2`, amb `WHERE sentence_id <> q` per excloure la pròpia frase: el vector no surt de la BD, com a P2, però sense escriure cap funció (14 línies de sistema a G2 davant de 71 a P2). Es mesuren dues configuracions sobre la mateixa taula: **exacta** (índexs desactivats, força bruta) i **HNSW** (aproximada); `EXPLAIN` comprova que cadascuna fa servir el pla esperat, i la configuració exacta ha de coincidir amb P2.

**Línies i crides.** G0 no té codi de sistema propi (és el de P0), G1 en té 52 (inclosa la creació dels índexs) i G2, 14, davant de les 71 de P2. Les crides són les de PostgreSQL, més dos `CREATE INDEX`.

### 3.2. Resultats i preguntes

**Estabilitat de les insercions.** Amb lots, desar vectors és estable (CV per lot 4 %); el text de G0 té un CV més alt (15 %) perquè els lots duren ~10 ms i un sol lot lent hi pesa molt. Com a PostgreSQL, el text depèn del nombre de crides i els vectors, de la conversió a text. A mida 1 (25,4 s amb `vector(384)` i 27,8 s amb `REAL[]`) la diferència no s'ha d'atribuir al tipus: una prova intercalant insercions d'una fila a totes dues bases de dades dona el mateix cost per fila, i el text de G0 i P0, amb codi idèntic, ja difereix un 9 % a aquesta mida. El fila a fila, amb 10.000 crides, és la mesura més sensible a la càrrega de la màquina.

| Mida de lot | Text (G0): total (s) | desv. entre càrregues | vector (G1): total (s) | desv. entre càrregues | Ref.: <code>REAL[]</code> (P1) |
|-------------|----------------------|-----------------------|------------------------|-----------------------|--------------------------------|
| 1           | 24,306               | 5,072                 | 25,375                 | 2,406                 | 27,779                         |

| Mida de lot | Text (G0): total (s) | desv. entre càrregues | vector (G1): total (s) | desv. entre càrregues | Ref.: <i>REAL[]</i> (P1) |
|-------------|----------------------|-----------------------|------------------------|-----------------------|--------------------------|
| 10          | 3,402                | 0,047                 | 5,426                  | 0,281                 | 6,800                    |
| 50          | 0,744                | 0,023                 | 3,231                  | 0,059                 | 3,810                    |
| 100         | 0,416                | 0,002                 | 2,960                  | 0,020                 | 3,410                    |
| 500         | 0,131                | 0,001                 | 2,556                  | 0,004                 | 2,926                    |
| <b>1000</b> | 0,103                | 0,001                 | 2,469                  | 0,046                 | 2,784                    |
| 2000        | 0,080                | 0,002                 | 2,371                  | 0,050                 | 2,733                    |

**Taula 8.** pgvector: temps **total** de càrrega de les 10.000 frases per mida de lot (mitjana i desviació entre 3 càrregues). L'última columna és **P1**, per comparar. En negreta, la mida triada.

| Operació                                  | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV   |
|-------------------------------------------|----|----------|----------|--------------|------------|------|
| Emmagatzematge del text (G0)              | 30 | 8,7      | 16,7     | 10,3         | 1,6        | 15 % |
| Generació d'embeddings (G1)               | 10 | 1.198,0  | 1.932,4  | 1.590,6      | 219,4      | 14 % |
| Emmagatzematge de <b>vector(384)</b> (G1) | 30 | 230,6    | 275,6    | 246,9        | 9,3        | 4 %  |
| Construcció de l'índex HNSW, L2 (G1)      | 3  | 693,7    | 748,0    | 712,7        | 30,6       | 4 %  |
| Construcció de l'índex HNSW, Cosinus (G1) | 3  | 672,4    | 723,6    | 695,7        | 25,9       | 4 %  |

**Taula 9.** pgvector: temps **per lot** de 1000 frases (10 lots × 3 càrregues; generació, 10 lots) i per **construcció completa** de cada índex (3 construccions).

**Estabilitat de les consultes i diferències entre mètriques.** Les dues mètriques donen el mateix top-2 (10/10) i compleixen la identitat ( $|L2^2 - 2d_{cos}| \leq 2 \cdot 10^{-7}$ ; aquí <-> retorna la distància L2 sense elevar al quadrat), amb temps equivalents. Les consultes exactes són estables (CV 10 %); amb HNSW el CV és més alt (18 % a L2), però sobre temps d'un mil·lisegon, on qualsevol interrupció del sistema pesa proporcionalment més.

| Configuració                  | n  | mín (ms) | màx (ms) | mitjana (ms) | desv. (ms) | CV   | × P2 | Recall vs P2 |
|-------------------------------|----|----------|----------|--------------|------------|------|------|--------------|
| Exacta (sense índex), L2      | 30 | 2,61     | 3,58     | 2,99         | 0,31       | 10 % | 93   | 100 %        |
| Exacta (sense índex), Cosinus | 30 | 2,63     | 3,78     | 3,04         | 0,29       | 10 % | 105  | 100 %        |
| HNSW, L2                      | 30 | 0,68     | 1,37     | 0,91         | 0,16       | 18 % | 305  | 100 %        |
| HNSW, Cosinus                 | 30 | 0,71     | 1,16     | 0,91         | 0,10       | 12 % | 353  | 100 %        |

**Taula 10.** G2: temps **per consulta** del top-2 (una sola crida SQL); n = 10 consultes × 3 rondes. × P2: vegades més ràpid que la força bruta en SQL de P2.

La dada més reveladora és la **cerca exacta**: fa la mateixa força bruta que P2, però amb la distància implementada en C dins el tipus, i triga 3,0 ms, **93 vegades menys**. Amb el mateix algorisme, el 99 % del temps de P2 ve d'haver d'expressar la distància en SQL interpretat sobre un tipus que la BD no entén, no de la cerca exhaustiva: és l'*impedance mismatch* mesurat. L'índex HNSW baixa a 0,9 ms (305× P2) amb *recall* del 100 % i *ef\_search* = 40.

**Millores possibles.** Ajustar *hnsw.ef\_search*, *mief\_construction* (temps per *recall*); *halfvec* (float16) per reduir a la meitat taula i índexs; *COPY* binari (amb *psycpg 3*) per evitar el literal de text; més *maintenance\_work\_mem* per construir l'índex; i, com que els vectors estan normalitzats, un sol índex de producte escalar (<#>) serviria les dues mètriques.

#### Diferències amb Chroma: pros i contres.

- *Model de dades.* pgvector afegeix el vector com una columna més d'una taula relacional: el text, les metadades i el vector conviuen amb SQL, transaccions, claus i **WHERE** natius, i una sola còpia de les dades serveix diverses mètriques.

A Chroma el vector és la unitat central, el text no pot existir sense ell i cada mètrica exigeix una col·lecció (i una còpia) nova.

- *Rendiment.* En aquest experiment pgvector amb HNSW respon 14 vegades més ràpid que Chroma amb el filtre d'exclusió, perquè el `WHERE` és un filtre SQL barat: `EXPLAIN` mostra que PostgreSQL recorre l'índex HNSW en ordre de distància i descarta la pròpia frase sobre la sortida de l'índex, sense cap filtre de metadades; sense filtre, els dos índexs queden en el mateix ordre de magnitud. Chroma s'estalvia la conversió a text dels vectors, però no n'és més ràpid en desar-los.
- *Contres de pgvector:* cal una extensió instal·lada al servidor, el conductor encara serialitza els vectors a text, i l'índex s'ha de crear, dimensionar i ajustar a mà. *Contres de Chroma:* sense transaccions ni SQL, dades duplicades per mètrica, i un filtre de metadades que aquí encareix la consulta; a favor, una API mínima (`add/query`) i la generació d'embeddings integrada.

## 4. Discussió

En les 10 consultes, els tres sistemes han retornat els mateixos top-2 (Taula següent; amb HNSW és un resultat observat, no una garantia): la diferència entre ells no és *què* responen, sinó quant costa arribar-hi i quanta feina s'ha d'afegir al sistema per fer-ho.

| Consulta | 1r veí | $d_{cos}$ | Text del 1r veí                                              | 2n veí | $d_{cos}$ |
|----------|--------|-----------|--------------------------------------------------------------|--------|-----------|
| 654      | 651    | 0,278     | jasons face twisted up and a snort of laughter fought its... | 708    | 0,285     |
| 1114     | 31     | 0,482     | out of my left eye i saw the outline of a creature .         | 1370   | 0,482     |
| 2025     | 2026   | 0,488     | it also involves a bad choice of a place where you roll a... | 2734   | 0,559     |
| 3759     | 2979   | 0,419     | he was trying to pretend it was just a piece of dust he h... | 2902   | 0,432     |
| 4281     | 5538   | 0,251     | wara felt sick , watching him go out of the corner of her... | 6098   | 0,256     |
| 5250     | 6094   | 0,276     | wara pressed her lips together , really beginning to wond... | 5102   | 0,293     |
| 6228     | 6218   | 0,348     | ` i do this as my jihad against the cause of injustice ,...  | 6220   | 0,421     |
| 7142     | 7119   | 0,450     | trae watched her for a moment , then became distracted as... | 7121   | 0,451     |
| 8754     | 9803   | 0,537     | im already at two hundred grand a hit , and that will mov... | 8751   | 0,551     |
| 9104     | 7892   | 0,463     | he then poured off some of the boiling water into third p... | 7922   | 0,469     |

**Taula 11.** Top-2 de cada consulta (ID = `sentence_id`). Idèntic a PostgreSQL pur (P2), Chroma (C2) i pgvector (G2) i per a les dues mètriques; es mostra la distància cosinus.

|                            | PostgreSQL pur              | pgvector                             | Chroma                              |
|----------------------------|-----------------------------|--------------------------------------|-------------------------------------|
| Element central            | la fila (text)              | la fila, amb el vector com a columna | el vector (el text n'és un atribut) |
| Tipus del vector           | <code>REAL[]</code> genèric | <code>vector(384)</code>             | vector de la col·lecció             |
| Com viatja el vector       | text ' <code>{...}</code> ' | text ' <code>[...]</code> '          | array numpy                         |
| On es calcula la distància | funcions SQL escrites a mà  | operadors natius (C)                 | índex HNSW                          |
| Índex vectorial            | cap de possible             | HNSW / IVFFlat, un per mètrica       | HNSW, sempre                        |
| Cerca                      | exacta                      | exacta o aproximada                  | aproximada                          |
| Dades per mètrica          | 1 taula                     | 1 taula + 1 índex per mètrica        | 1 col·lecció (còpia) per mètrica    |
| Transaccions i SQL         | sí                          | sí                                   | no                                  |

|                                                             | PostgreSQL pur | pgvector             | Chroma                  |
|-------------------------------------------------------------|----------------|----------------------|-------------------------|
| Línies de sistema (text / emb. / cerca)                     | 27 / 40 / 71   | 0 / 52 / 14          | 23 / 17 / 12            |
| Crides per consulta                                         | 1              | 1                    | 2 (get + query)         |
| Desar el text: total de la càrrega, lots de 1000 (s)        | 0,10           | 0,10                 | 141,3 (amb embeddings)  |
| Desar els embeddings: total de la càrrega, lots de 1000 (s) | 2,78           | 2,47                 | 2,97                    |
| Consulta top-2, L2: per consulta (ms)                       | 277,1          | 3,0 exacta, 0,9 HNSW | 12,9 (0,9 sense filtre) |

**Taula 12.** Els tres sistemes des de l'*impedance mismatch*. Línies de sistema: mateix criteri a tots (només codi que parla amb el sistema, sense validacions ni `common.py`); a Chroma, la càrrega del text inclou la generació d'embeddings. Temps: mitjana de les 3 càrregues (o de les consultes) de les seccions anteriors.

**Una gradació de l'*impedance mismatch*.** Les dades mostren tres graus d'encaix entre la naturalesa de les dades i la representació interna de cada sistema. A **PostgreSQL pur** el desajust és el més gran dels tres: el vector és un array genèric que la base de dades no tracta com a vector, i tot el que el fa útil (la distància, el rànk, l'exclusió) s'ha d'escriure a mà en SQL interpretat, amb un cost en línies de codi i en temps: la mateixa força bruta que pgvector fa en 3,0 ms hi triga 277 ms. **pgvector** tanca el desajust dins el mateix motor: el vector passa a ser un tipus amb operadors i índexs, i el model relacional (taules, SQL, transaccions, una sola còpia de les dades) es conserva intacte; només queda el desajust a la frontera amb el client, que continua enviant els vectors com a text. **Chroma** redueix fortament el desajust vectorial, perquè el vector és la seva unitat i la cerca vectorial forma part del seu model (tot i que encara cal una crida per obtenir el vector d'una frase abans de cercar), però a canvi perd el model relacional: el text no pot existir sense vector, cada mètrica exigeix una còpia de les dades, no hi ha transaccions ni SQL, i l'exclusió d'una frase, trivial amb un `WHERE`, passa a ser un filtre de metadades que aquí multiplica per 14 el temps de consulta.

**PostgreSQL vs. Chroma.** *PostgreSQL pur*: a favor, un sol sistema per a dades i vectors, transaccions, SQL i resultats exactes; en contra, cap tipus ni índex vectorial, més codi i consultes entre 302 i 367 vegades més lentes que amb un índex vectorial (pgvector amb HNSW o Chroma sense filtre, Taules 7 i 10), que en ser força bruta creixen amb la mida del corpus. *Chroma*: a favor, una API mínima, índex vectorial des del primer moment i generació d'embeddings integrada; en contra, tot el que no és vector (text, metadades, consistència) queda subordinat al vector, i el que no encaixa amb el seu model s'ha de resoldre fora. *pgvector* ocupa el punt mig: en aquest experiment combina tipus, operadors i índexs vectorials nadius amb el model relacional, i redueix gran part del desajust sense separar els vectors de la resta de dades (no s'hi han avaluat concurrència, volums més grans ni cost d'operació). La lliçó general coincideix amb la de l'enunciat: com més s'ajusta la representació interna a la naturalesa de les dades, menys codi i adaptacions calen, i en aquest experiment també menys temps de consulta.

## 5. I am an AI agent. Tell me what I have to build.

**Rols.** Claude Code (Claude Opus 5.5), amb accés al repositori, va implementar i executar els scripts i va redactar l'informe, que revisàvem nosaltres; altres xats (Claude, Gemini, ChatGPT) van fer de revisors independents del document. Les decisions de disseny les vam prendre nosaltres: la IA proposava alternatives amb els seus costos i nosaltres triàvem (p. ex. el disseny de C0/C1, les repeticions de Chroma o fer pgvector).

**Raonament de les instruccions.** En lloc de prompts llargs, vam mantenir un context persistent al repositori que la IA llegeix a cada sessió: un `CLAUDE.md` amb les decisions ja preses (marcades per no tornar-les a discutir), les indicacions del professor (insercions per lots, distàncies dins la BD) i els criteris de mesura, i l'enunciat literal com a font de veritat. Així, cada prompt podia ser una tasca curta ("fes P0") i les respostes eren coherents entre sessions i entre els dos membres de l'equip. Tres regles hi van pesar especialment: cap xifra del document escrita a mà (tot surt dels `results/*.json`), totes les mesures oficials en una sola màquina, i un registre de cada decisió en què intervenia la IA (`docs/ai_log.md`), que és la font d'aquesta secció.

**Com es van refinar.** Les instruccions van créixer a partir dels errors de la IA. En desar l'enunciat sencer com a font de veritat vam veure que el C1 que havia dissenyat a partir d'un resum no complia "generate and store", i es va redissenyar; quan va ajornar la redacció de [PQ1], vam afegir la regla d'escriure les observacions en acabar cada script; quan va

proposar retallar el grid de **C0** per estalviar temps, la vam corregir amb la regla del “grid únic” que ella mateixa havia documentat. Altres decisions van canviar en veure les dades: l’equivalència  $L2/\cos$  es va documentar i verificar en lloc de canviar de mètrica (Chroma no admet  $L1$ ), i la mida de lot es va justificar pel comportament del sistema i no per conveniència estadística.

**Com es va validar.** No vam acceptar cap resultat només perquè la IA el donés per bo. Cada script porta comprovacions independents: vectors recuperats bit a bit, **P2** contrastat amb numpy (només com a oracle), **EXPLAIN** dels plans, *recall* contra **P2**, la identitat  $L2^2 = 2 - 2 \cdot \cos$  a les dades i una taula del top-2 que fa fallar el document si els sistemes no coincideixen. Les anomalies es van investigar en lloc d’explicar-les: una latència de ~45 ms per missatge resultava ser Docker per IPv6 i no PostgreSQL; el cost de desar **REAL[]** era la conversió float  $\rightarrow$  text; una diferència al fila a fila de pgvector es va comprovar que era soroll amb una prova intercalada. Finalment, les crítiques dels revisors externs també es van verificar contra les dades abans d’aplicar-les: n’hi va haver d’encertades (la comparació justa de **C0** era amb **P0** més la generació de **P1**, i la conclusió es va invertir), i d’errònies (un percentatge calculat amb valors arrodonits, una lectura del PDF que no coincidia amb el PDF real, o suggeriments per a aquesta mateixa secció que descrivien un ús de la IA que no havíem fet). La lliçó és que la IA ens va fer molt més ràpids, però la fiabilitat va venir de comprovar-ho tot amb execucions i dades.